# Forensic USB Drive Acquisition

A court-admissible, bit-for-bit imaging workflow for preserving digital evidence in insider threat investigations.

# Incident Overview

A corporate investigation has identified a suspected insider who exfiltrated sensitive financial documents from the organisation's network. Evidence indicates the employee used a USB removable storage device to copy proprietary spreadsheets containing confidential financial data.

The forensic mandate is twofold: acquire the USB drive contents without altering the original media, and produce a verified bit-for-bit forensic image whose integrity can withstand courtroom scrutiny. Every procedural step must be documented to satisfy chain-of-custody requirements and demonstrate admissibility under applicable evidentiary standards.

**Legal Imperative:** The investigator must demonstrate through cryptographic hashing that the forensic copy is an exact replica of the original — any deviation renders the image inadmissible.

# Technical Environment

## Suspect Device

USB flash drive containing financial spreadsheets. Serial number, make, and model must be recorded prior to any interaction with the media.

## Forensic Workstation

Linux or Windows system equipped with validated imaging tools (FTK Imager, dc3dd, Guymager). The workstation must be forensically clean and auditable.

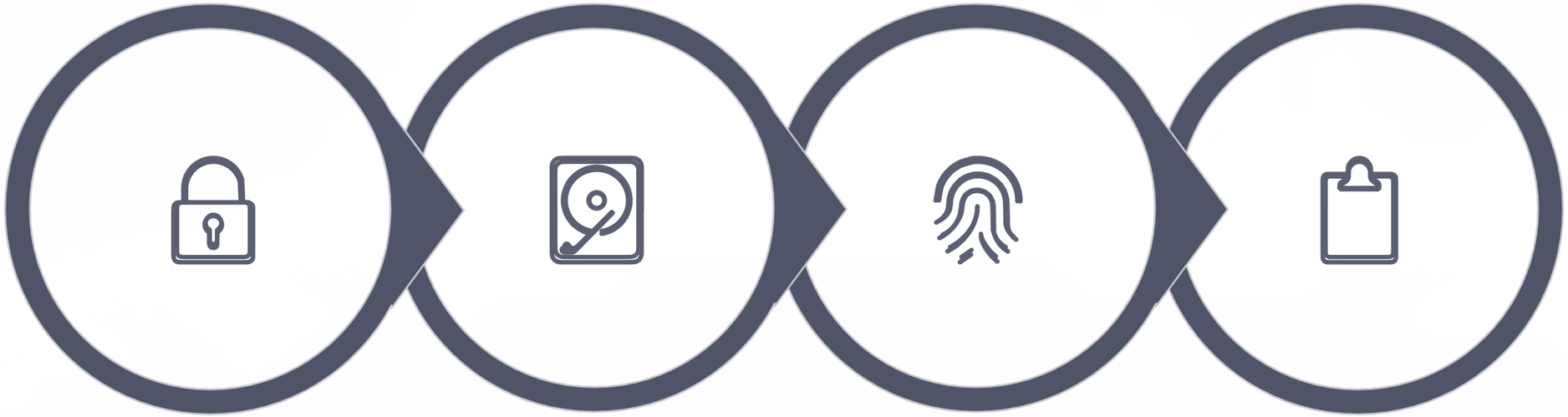
## Hardware Write-Blocker

A certified hardware write-blocker prevents any write commands from reaching the suspect USB, preserving original evidence in its pristine state.

## Forensic Storage

External forensic-grade HDD designated exclusively for storing acquired images, hash logs, and acquisition documentation — never the source device.

# Four-Phase Forensic Process



Read-Only Connect

Bit-by-Bit Image

Hash Verification

Documentation

Each phase builds upon the previous one, forming an unbroken evidentiary chain from initial device contact through to court-ready documentation. Skipping or misordering any phase jeopardises admissibility.

# Preparation & Read-Only Connection

**Objective:** Prevent any modification to the original USB drive before, during, and after the acquisition process.

1

### Connect via Write-Blocker

Attach the suspect USB to the forensic workstation exclusively through a **certified hardware write-blocker**. This intercepts all write commands at the hardware level, ensuring zero alteration to the source media.

2

### Verify Detection

Confirm the operating system detects the device **without mounting it** in read-write mode. On Linux, use `lsusb` and `udevadm info`. On Windows, use FTK Imager or Magnet ACQUIRE to enumerate connected devices.

3

### Record Device Metadata

Document the device ID, serial number, make/model, and physical condition. This metadata anchors the evidence to a specific physical artefact throughout the legal process.

# Create Bit-by-Bit Forensic Image

**Objective:** Generate a faithful, exact replica of the USB contents — including slack space, deleted files, unallocated clusters, and hidden sectors — onto separate forensic storage.

## Imaging Tools

- **FTK Imager** — GUI/CLI, supports .E01 (EnCase) and raw .dd formats
- **dc3dd** — Linux CLI with embedded hashing and logging
- **ddrescue** — Resilient to read errors on damaged media
- **Guymager** — Linux GUI, supports parallel hashing during acquisition

## Key Parameters

- `bs=4M` — Block size setting accelerates imaging throughput
- `conv=sync,noerror` — Pads read errors with nulls and continues
- Enable **hash generation during acquisition** to capture integrity at the point of creation
- Always specify destination on a **separate forensic-grade device**

# Imaging Command Examples

The following Linux commands demonstrate two approaches to creating a verified forensic image. Both produce a bit-for-bit copy with cryptographic hash verification.

### Standard dd + SHA-256

Create the image, then hash separately:

```
sudo dd if=/dev/sdb of=/mnt/forensic/usb_image.dd bs=4M
status=progress conv=sync,noerror
```

```
sha256sum /mnt/forensic/usb_image.dd >
/mnt/forensic/usb_image.dd.sha256
```

### dc3dd with Embedded Hash

Hash is computed during acquisition and logged automatically:

```
sudo dc3dd if=/dev/sdb of=/mnt/forensic/usb_image.dd hash=sha256
log=/mnt/forensic/dc3dd_log.txt
```

**Best Practice:** dc3dd is preferred for forensic work because it embeds the hash in the acquisition log, reducing the risk of a detached hash file being lost or mislabelled.

# Hash Verification — Proving Bit-for-Bit Fidelity

**Objective:** Confirm the acquired image is a mathematically exact duplicate of the original USB device. This is the cornerstone of forensic integrity.

1

## Hash the Original

Compute the SHA-256 hash of the source USB (read-only, unmounted):

```
sudo sha256sum /dev/sdb > original_usb.sha256
```

2

## Hash the Image

Compute the SHA-256 hash of the acquired forensic image:

```
sha256sum /mnt/forensic/usb_image.dd > image_hash.sha256
```

3

## Compare & Verify

Run `diff original_usb.sha256 image_hash.sha256` — hashes **must match exactly**. Any discrepancy requires complete re-imaging.

Recommended tools include **FTK Imager**, **HashCalc**, and **md5sum / sha256sum** command-line utilities. Dual hashing with both MD5 and SHA-256 is strongly recommended for court submissions.

# Documentation & Chain-of-Custody

**Objective:** Maintain legal credibility by creating a comprehensive, tamper-evident record of every action taken during the acquisition process.

### Temporal & Identity Records

Date and time of acquisition (with timezone), operator name and initials, physical location of examination, and all personnel present during the process.

### Device & Tool Metadata

Device ID, serial number, make/model. Imaging tool name and exact version. Write-blocker manufacturer and model. Forensic storage device identification.

### Cryptographic Evidence

Hashing algorithm(s) used — both **MD5 and SHA-256** are recommended. Hash values of both the original device and the forensic image, recorded side-by-side for comparison.

### Secure Storage

All logs, hash files, and acquisition records must be stored securely alongside the forensic image. Any gap or discrepancy in documentation can render the entire image inadmissible.

# Critical Pitfalls & Mitigation Strategies

| Pitfall | Explanation | Mitigation |
|---|---|---|
| Normal USB Mounting | Updates timestamps, access logs, and metadata — silently corrupting the evidence | Always use a hardware write-blocker; never open or browse files directly |
| Single Hash Algorithm | MD5 alone has known collision vulnerabilities, reducing evidentiary weight | Dual hash with both **MD5 + SHA-256** for every acquisition |
| Interrupted Imaging | Power loss or incorrect block size produces an incomplete, unverifiable image | Use dc3dd with `conv=noerror`; monitor progress continuously |
| Insufficient Documentation | Missing timestamps, tool versions, or operator details undermine admissibility | Maintain a complete chain-of-custody log from first contact to final storage |
| Writing Image to Source | Overwrites and permanently destroys the original evidence | Always image to a **separate** forensic-grade storage device |

# For Court-Admissible Acquisition...

### Protect the Original

Hardware write-blockers are non-negotiable. No exceptions.

### Image Everything

Bit-for-bit copies capture slack space, deleted files, and hidden sectors.

### Verify with Hashes

Dual-algorithm hashing (MD5 + SHA-256) proves exact fidelity.

### Document Relentlessly

Every action, timestamp, and tool version must be recorded for the chain of custody.

> A forensic image is only as strong as the process that created it. Rigorous methodology — not just technology — is what withstands cross-examination.